

FILE ORGANIZATION PROJECT REPORT

1. Introduction

The aim of this project is to clean a large-scale password dataset, convert it into a standard format, and then store it on the disk using three different data storage methods (Plain Text, Hash, and Compressed).

This study aims to practically examine the Space-Time Trade-off concept, which is commonly seen in modern databases and file systems. The effects of storing data in different formats on disk space, search performance, and CPU/memory load were measured. The advantages and disadvantages of these methods were compared and analyzed against each other.

2. Tools and Technologies Used

The project was developed using the C++ language in a completely modular structure. The MSYS2 (MinGW-w64) compiler environment was preferred for managing external library dependencies.

- **Development Environment:** Visual Studio Code, g++ (GCC) Compiler
- **Zlib Library (<zlib.h>):** Used for data compression and decompression operations.
- **OpenSSL Library (<openssl/evp.h>):** Used to create MD5 hashes by applying one-way encryption to the passwords (The current EVP API was preferred).
- **Standard C++ Libraries:** <chrono> was used for time measurement, <fstream> for file operations, and the Windows API <psapi.h> for measuring RAM consumption.

3. Project Design

To protect data integrity and ensure maintainability, the project architecture was designed using folders and C++ modules separated by their tasks:

- **Folder Structure:** Consists of **RawData** (Raw data), **Processed** (Cleaned single reference data), **Storage** (Folders for different storage methods: Raw, Hashed, Compressed), and the **Code** folder containing the source codes.
- **Modular Code Architecture:**
 - **main.cpp:** Manages the main control flow and performance tests of the program.
 - **preprocess.cpp:** Reads the raw data, cleans it, and prevents duplicates.
 - **storage.cpp:** Writes the processed data to the disk in Raw, Hashed, and Compressed formats.
 - **search.cpp:** Performs search tests on 100 random passwords.
 - **compression.cpp:** Handles read/write operations using zlib.
 - **utils.cpp:** Contains helper tools like MD5 calculation, file size measurement, and RAM tracking.

4. Data Preprocessing

The raw password file (`passwords.txt`) obtained from the SecLists repository was not used directly. It was transferred to the `Processed` folder as a standardized reference.

- When reading the data, hidden carriage return (`\r`) characters left at the end of the lines were removed, and empty lines were skipped.
- **Removing Duplicate Records:** To ensure memory and time efficiency, the `std::unordered_set` data structure, which has an $O(1)$ search cost, was used. Passwords that could not be added to the set (because they already exist) were skipped, ensuring the file consists of completely unique records.
- **Hash Generation:** Data was converted into a 32-character hexadecimal string format using OpenSSL's `EVP_Digest` structure for MD5 generation. Case normalization (uppercase/lowercase) was not applied to avoid ruining the originality (case-sensitivity) of the data.

5. Data Storage Methods

To protect data integrity, all storage operations were performed using only the clean data in the `Processed` folder:

- **Raw Storage:** The Processed data was directly written to a `.bin` file without any transformation. This method serves as the baseline reference point for performance measurements.
- **Hashed Storage:** Before being written to the disk, the data was passed through the `utils::calculateMD5()` function, and only these MD5 values were written to the `.bin` file instead of the actual password text.
- **Compressed Storage:** The Processed data was compressed in binary format and written using the `gzopen` and `gzputs` functions of the `zlib` library.

6. Search and Performance Measurements

- **Search Method:** Since the data is not sorted, the search operation was done using the Linear Search ($O(N)$) logic by reading the file from beginning to end (`getline`). For performance measurement, 100 random passwords were selected from the Processed data and searched in each method.
- **Time and Disk Measurements:** Time measurements were made in microseconds (μs) using `std::chrono::high_resolution_clock`, and their standard deviations were calculated. Disk sizes were measured in bytes using C++'s end-of-file cursor reading method `tellg()`.
- **CPU and RAM:** During the operations, a single core ran at 100% load. The RAM usage was measured with `GetProcessMemoryInfo` and stayed around ~8.2 MB, proving that the entire data was processed using a streaming method rather than loading it all into memory at once.

7. Results and Evaluation

The performance metrics of the 100 random searches obtained at the end of the project are summarized in the table below:

Method	Size (Bytes)	Compression Ratio	Avg. Time (µs)	Max Time (µs)	Decomp. Time (µs)
Raw	881,877	-	1609.22	3369	-
Hashed	3,399,966	-	2594.58	5240	-
Compressed	373,167	0.42	1792.68	19,832	19,792

Analysis and Comparison:

- **Space Saving:** The Compressed method was by far the most successful in disk usage, reducing the data size by 58% (0.42 ratio). The Hashed method created the biggest disadvantage in disk space by increasing the file size by about 4 times, as it produces a fixed 32-character output.
- **Speed and Search Efficiency:** Raw Storage is the fastest method with an average of 1.6 ms because it performs direct text comparison during the search.
- **Compression and Hash Impact:** In the hashed search, converting the target password to MD5 instantly (CPU cost) and the cost of reading the disk due to the large file size increased the time. However, since the original password is not stored as text, it provides very strong data security that makes up for this time loss.
- **Bottleneck:** To search within the compressed data, the file first had to be decompressed using zlib. Since the decompression alone took ~20 ms (as seen from the maximum time of 19,832 µs), it created a huge CPU and time cost for the search operation.

General Evaluation: If high security is required in the system, Hashed should be preferred by sacrificing space; if searches will be rare and disk space needs to be saved, Compressed should be preferred by sacrificing time; if search speed is critical, the Raw storage method should be preferred by sacrificing security.

8. References

- SecLists Password Repository: <https://github.com/danielmiessler/SecLists>
- C++ Standard Library Documentation: <https://cplusplus.com/>
- OpenSSL EVP Library Documentation
- Zlib Compression Algorithm Guide